Buổi 3: Xử lý sự kiện người dùng

Nội dung chính

3.1. Sử dụng các thành phần View

3.2. Resource trong ứng dụng Android

3.3. Xử lý sự kiện người dùng

Mục tiêu cần đạt được

Hiểu cách sử dụng các thành phần View như TextView, Button, EditText, CheckBox, RadioButton, Spinner và AutoCompleteTextView để tạo giao diện người dùng tương tác trên Android.

Biết cách quản lý tài nguyên trong ứng dụng Android thông qua các tệp tài nguyên như chuỗi ký tự, màu sắc, hình ảnh, và bố cục, từ đó dễ dàng tùy chỉnh và hỗ trợ đa nền tảng.

Vận dụng các thành phần View và tài nguyên vào trong các ứng dụng thực tế, bao gồm việc xây dựng giao diện, xử lý sự kiện và tối ưu hóa trải nghiệm người dùng.

3.1. Sử dụng các thành phần View

3.1.1. Đối tượng TextView

Đối tượng TextView thường được sử dụng để miêu tả hoạt động trên giao diện hoặc hướng dẫn người dùng nhập liệu.

Các TextView được khai báo trong file XML cùng với một đối tượng layout như sau:

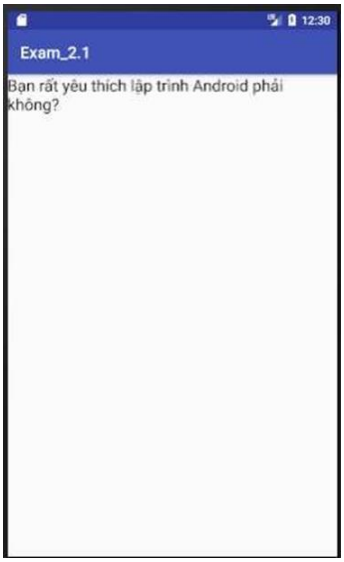
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="You were expecting something profound?" />
</LinearLayout>
```

Thuộc tính android:text chứa giá trị người lập trình muốn hiển thị trên TextView. Các thuộc tính khác của đối tượng TextView thường được sử dụng là:

- android:typeface thiết lập bề mặt chữ (các giá trị như normal, sans, serif, monospace).
- android:textStyle xác định kiểu chữ như đậm (bold), nghiêng (italic), hoặc vừa nghiêng vừa đậm (bold_italic).

- android:textColor thiết lập màu chữ, có thể nhận các giá trị ở hệ hexa từ #000000 đến #FFFFFF hoặc viết ở dạng các hằng xâu chỉ màu sắc như "red", "green", . . .

Ví dụ tfhiết kế giao diện cho ứng dụng Android gồm một TextView hiển thị dòng chữ như hình dưới.



Hình 3.1: Ví dụ về đối tượng TextView

File activity_main.xml có nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:text="Bạn rất yêu thích lập trình phải không?" />
</LinearLayout>
```

3.1.2. Đối tượng Button

Đối tượng Button (nút nhấn) cho phép người dùng nhấn chọn trên giao diện để thực hiện một công việc cụ thể theo yêu cầu của bài toán. Trên cùng một giao diện ứng dụng có thể có nhiều Button khác nhau. Khai báo đối tượng Button trong file XML như sau:

```
<Button
    android:id="@+id/start_button"
    android:text="@string/Start"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

Đối tượng Button trên có định danh (id) là start_button và có nhãn hiện thị với người dùng là biến xâu có id là Start. Chiều rộng và chiều cao của Button sẽ tự động tăng lên hoặc giảm xuống theo kích thước của nhãn hiển thị trên nó. Để xử lý sự kiện trên các

Button, từ chương trình java cần lấy về định danh của chúng sử dụng phương thức findViewById().

Hình 3.2: Ví dụ về đối tượng Button

3.1.3. Đối tượng EditText

Đối tượng TextView chỉ cho phép người dùng đọc dữ liệu trên giao diện, còn đối tượng EditText cho phép người dùng vừa dùng vừa đọc, vừa ghi dữ liệu lên nó. Đối tượng EditText được khai báo trong file xml như sau:

```
<EditText
    android:id="@+id/field"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:singleLine="false" />
```

Các thuộc tính thường sử dụng của đối tượng EditText bao gồm:

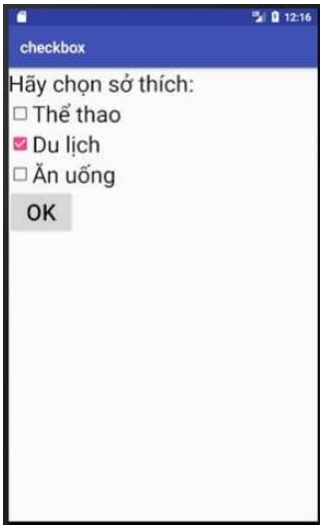
- android:autoText: cho phép tự động kiểm tra lỗi chính tả khi nhận giá trị true.
- android:capitalize: tự động chuyển chữ cái đầu tiên thành chữ in hoa khi nhận giá trị true.
- android:digits: chỉ cho phép nhập ký tự số khi nhận giá trị true.
- android:singleLine: chỉ cho phép nhập ký tự trên một dòng khi nhận giá trị false.
- android:maxLength: nhận giá trị là một số cụ thể đặt trong cặp "" để thiết lập số lượng kí tự tối đa nhập vào.
- android:inputType: nhận các giá trị "number" hoặc "text" cho phép chỉ định kiểu dữ liệu được nhập vào.
- android:password: nhận giá trị true cho phép người dùng nhập dữ liệu dưới dạng mật khẩu (password).
- android:imeOptions: cho phép người dùng nhấn Enter tương ứng với nhấn nút Next trên bàn phím mềm để chuyển đến đối tượng nhập liệu tiếp theo trên giao diện.

Hình 3.3: ví dụ về đối tượng EditText

3.1.4. Đối tượng CheckBox

Khi người lập trình muốn thiết kế giao diện cho phép người sử dụng lựa chọn các giá trị cố định trên giao diện thì có thể sử dụng một trong các đối tượng như CheckBox hoặc Radio Button hay Spinner. Đối tượng Radio Button và Spiner sẽ được trình bày ngay trong các mục tiếp theo.

Các đối tượng CheckBox cho phép người dùng lựa chọn một hoặc nhiều lựa chọn tuỳ ý. Chẳng hạn, trên giao diện liệt kê các sở thích "Thể thao", "Du lịch", "Ăn uống", người dùng có thể lựa chọn nhiều sở thích bằng cách chọn nhiều CheckBox trên giao diện. Khai báo đối tượng CheckBox trong file xml như sau:



Hình 3.4: Minh hoạ đối tượng CheckBox.

```
<CheckBox
    android:id="@+id/cbxBox1"
    android:layout_width="20dp"
    android:layout_height="20dp"
    android:text="CheckBox" <!-- nhãn miêu tả -->
    android:checked="false" />
```

Thuộc tính android:checked cho phép thiết lập trạng thái của CheckBox là true (checked) hoặc false (unchecked).

Các thuộc tính thường dùng của CheckBox trong chương trình java là: .isChecked(): trả về giá trị 'true' nếu CheckBox ở trạng thái được chọn, ngược lại trả về giá trị false .setChecked(): thiết lập trạng thái checked hoặc unchecked cho CheckBox

3.1.5. Đối tượng Radio Button

Các đối tượng RadioButton thông thường được nhóm theo từng nhóm để cho phép người dùng chỉ được chọn một lựa chọn trong mỗi nhóm. Chẳng hạn, khi chọn giới tính thì thông thường người dùng được phép chọn một trong hai giá trị 'Nam' hoặc 'Nữ'.

Đối tượng RadioGroup nhóm các đối tượng RadioButton theo tường nhóm. Khai báo đối tượng RadioButton trong file XML như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="Hãy chọn một hệ điều hành:" />

    <RadioGroup
        android:id="@+id/group"
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <RadioButton
            android:id="@+id/radio1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="30sp"
            android:text="Android" />

        <RadioButton
            android:id="@+id/radio2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="30sp"
            android:text="iOS" />

        <RadioButton
            android:id="@+id/radio3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="30sp"
            android:text="WindowsPhone" />
    </RadioGroup>

    <TextView
```

```
        android:id="@+id/tv"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="" />
</LinearLayout>
```

Hình 3.5: Ví dụ về đối tượng RadioButton

3.1.6. Đối tượng Adapter

Đối tượng Adapter thường được sử dụng kết hợp với các đối tượng AutoCompleteTextView, Spinner và Listview để cung cấp dữ liệu cho các đối tượng này. Trong các mục tiếp theo, đối tượng Adapter sẽ được trình bày cụ thể với từng trường hợp nêu trên.

Đối tượng Adapter được sử dụng nhiều nhất là ArrayAdapter. Mỗi đối tượng ArrayAdapter khi được khởi tạo cần ba tham số. Đó là ngữ cảnh (context), kiểu phần tử và danh sách các phần tử. Thông thường ngữ cảnh là chỉ chính lớp chứa nó, kiểu phần tử thường được chọn là một trong các kiểu có sẵn của Android theo cú pháp android.R.layout.tên_định_danh, chẳng hạn android.R.layout.simple_list_item_1 hoặc an- droid.R.layout.simple_list_item_2,. . . . Danh sách các phần tử thường là một danh sách các hằng số được định nghĩa theo một số cách khác nhau. Ví dụ:

Cách 1: String[] items={"this", "is", "a", "really", "silly", "list"};

Cách 2: List<String> items = new ArrayList<String>();

Sau đó thêm các phần tử bằng phương thức add(), ví dụ: items.add("Athos");

items.add("Porthos"); items.add("Aramis");

Đối tượng ArrayAdapter sẽ được khởi tạo như sau: ArrayAdapter<String> adt = new ArrayAdapter<String>(this, android.R.layout.simple_list_item_1, items);

Ngoài ra, người lập trình có thể đọc thêm về hai đối tượng Adapter khác là CursorAdapter để chuyển đổi con trỏ và Sim- pleAdapter để chuyển đổi dữ liệu từ các file XML.

3.1.7. Đối tượng Spinner

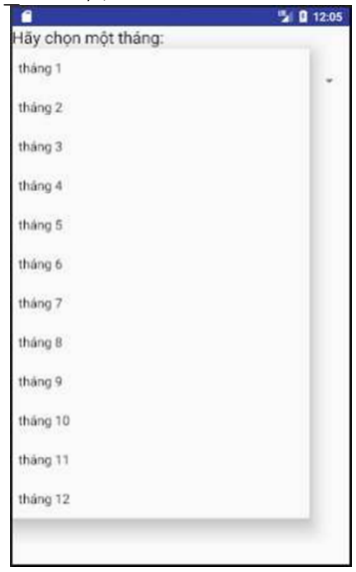
Đối tượng Spinner có chức năng giống như đối tượng ComboBox trong lập trình GUI nói chung.

Đối tượng Spinner được định nghĩa trong file XML như sau:

```
<Spinner
      android:id="@+id/spinner"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content" />
```

Dữ liệu của mỗi đối tượng Spinner thường được lấy từ một đối tượng ArrayAdapter sử dụng các phương thức sau với adt là một Spinner:

```
adt.setDropDownViewResource(android.R.layout.
simple_spinner_dropdown_item);
```



Hình 3.6: Minh họa đối tượng Spinner

3.1.8. Đối tượng AutoCompleteTextView

Đối tượng AutoCompleteTextView cho phép người dùng nhập dữ liệu nhanh và thuận tiện hơn đối tượng EditText, nó tự động hiển thị danh sách các từ gợi ý để người dùng lựa chọn dựa trên các ký tự đầu tiên được nhập bởi người dùng. Khai báo đối tượng AutoCompleteTextView trong file xml như sau:

```
<AutoCompleteTextView
    android:id="@+id/autocomplete"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:completionThreshold="2" />
```

Thuộc tính completionThreshold cho phép đặt số lượng các ký tự đầu tiên được so khớp giữa các từ trong thư viện được với từ đang được nhập trong AutoCompleteTextView.

3.1.9. Tạo giao diện hoàn thiện

Kết hợp các thành phần View trong Android là một yếu tố quan trọng trong việc xây dựng giao diện người dùng hoàn chỉnh và dễ dàng tương tác. Các thành phần View

cơ bản như TextView, ImageView, Button, EditText và RecyclerView có thể được kết hợp với nhau để tạo ra những giao diện ứng dụng phong phú và trực quan.

Để tổ chức các View này một cách hợp lý, Android cung cấp nhiều loại layout khác nhau, mỗi loại layout phục vụ một mục đích nhất định. Ví dụ, LinearLayout sắp xếp các thành phần theo chiều dọc hoặc chiều ngang, giúp tạo ra một cấu trúc đơn giản và dễ hiểu. RelativeLayout cho phép bố trí các View dựa trên mối quan hệ vị trí giữa chúng, trong khi ConstraintLayout cung cấp sự linh hoạt cao hơn với các ràng buộc (constraints), cho phép bạn kiểm soát vị trí của các View trong giao diện một cách chi tiết.

Khi thiết kế giao diện, việc lựa chọn layout phù hợp là rất quan trọng. Mỗi layout có những ưu và nhược điểm riêng, và tùy thuộc vào yêu cầu cụ thể của ứng dụng, một layout có thể phù hợp hơn so với các layout khác. Ví dụ, trong các ứng dụng có nội dung dài, ScrollView có thể được sử dụng để làm cho giao diện có thể cuộn được, đảm bảo người dùng có thể truy cập vào toàn bộ nội dung trên màn hình.

Trong khi đó, với các danh sách dữ liệu lớn, việc sử dụng RecyclerView là lựa chọn tối ưu. RecyclerView giúp tiết kiệm bộ nhớ và cải thiện hiệu suất của ứng dụng, vì nó chỉ tải các phần tử của danh sách khi người dùng cuộn đến chúng, thay vì tải tất cả các phần tử cùng một lúc.

Một yếu tố quan trọng khác trong việc kết hợp các thành phần View là sự chú ý đến trải nghiệm người dùng. Các thành phần như Button, TextView, và EditText không chỉ cung cấp các chức năng cơ bản mà còn cần được thiết kế sao cho người dùng dễ dàng nhận biết và sử dụng. Việc điều chỉnh kích thước, màu sắc, vị trí, cũng như sử dụng các hiệu ứng chuyển động có thể tạo ra một giao diện người dùng sinh động và thu hút.

Kết hợp một cách hợp lý các thành phần View và sử dụng các layout thích hợp sẽ giúp xây dựng một giao diện người dùng vừa đẹp mắt vừa hiệu quả, đáp ứng nhu cầu của người sử dụng và tối ưu hóa hiệu suất ứng dụng trên các thiết bị Android khác nhau.

3.2. Resource trong ứng dụng Android

Quản lý tài nguyên (resources) trong Android là một phần quan trọng trong việc phát triển ứng dụng, vì nó cho phép ứng dụng sử dụng các tài nguyên như hình ảnh, chuỗi ký tự, màu sắc, bố cục, và các giá trị khác một cách linh hoạt và dễ dàng quản lý. Dưới đây là các loại tài nguyên chính và cách quản lý chúng trong Android:

3.2.1. Chuỗi ký tự (String Resources)

Chuỗi ký tự được định nghĩa trong tệp res/values/strings.xml. Điều này cho phép dễ dàng thay đổi ngôn ngữ hoặc sửa nội dung mà không cần thay đổi mã nguồn.

Ví dụ định nghĩa:

```
xml
Copy code
<resources>
    <string name="app_name">My Application</string>
    <string name="welcome_message">Welcome to the app!</string>
</resources>
```

Sử dụng trong mã Java/Kotlin:

```
val message = getString(R.string.welcome_message)
```

3.2.2. Màu sắc (Color Resources)

Màu sắc được định nghĩa trong tệp res/values/colors.xml. Điều này giúp quản lý màu sắc ứng dụng một cách nhất quán.

Ví dụ định nghĩa:

```
xml
Copy code
<resources>
    <color name="primaryColor">#FF0000</color>
    <color name="secondaryColor">#00FF00</color>
</resources>
```

Sử dụng trong mã:

```
val color = ContextCompat.getColor(context, R.color.primaryColor)
```

3.2.3. Hình ảnh (Drawable Resources)

Hình ảnh được lưu trữ trong thư mục res/drawable/. Android hỗ trợ nhiều định dạng hình ảnh như .png, .jpg, và .xml (Vector Drawable).

Sử dụng trong mã:

```
val image = ContextCompat.getDrawable(context, R.drawable.my_image)
```

3.2.4. Bố cục (Layout Resources)

Bố cục (layout) định nghĩa cách sắp xếp các thành phần giao diện người dùng (UI) và được lưu trong thư mục res/layout/. Các tệp bố cục thường có định dạng .xml.

Ví dụ bố cục:

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:text="@string/welcome_message"/>

</LinearLayout>
```

- Sử dụng trong mã:

```
setContentView(R.layout.activity_main)
```

3.2.5. Giá trị (Values Resources)

Ngoài chuỗi và màu sắc, ta còn có thể định nghĩa các tài nguyên khác như kích thước (dimensions), kiểu chữ (styles), và mảng (arrays) trong tệp res/values/.

Ví dụ kích thước (dimensions):

```
xml
Copy code
<resources>
    <dimen name="text_size">16sp</dimen>
</resources>
```

Sử dụng trong mã:

```
val textSize = resources.getDimension(R.dimen.text_size)
```

3.2.6. Tài nguyên nhiều ngôn ngữ và màn hình

Ta có thể tạo các tệp tài nguyên riêng biệt cho từng ngôn ngữ hoặc kích thước màn hình khác nhau bằng cách thêm các hậu tố vào tên thư mục tài nguyên. Ví dụ:

res/values-en/strings.xml cho ngôn ngữ tiếng Anh.

res/values-v21/colors.xml cho các thiết bị chạy Android API 21 trở lên.

3.2.7. Sử dụng tài nguyên với Data Binding và Compose

Nếu bạn sử dụng Data Binding hoặc Jetpack Compose, việc truy cập tài nguyên sẽ có sự khác biệt. Ví dụ, trong Jetpack Compose, ta có thể dùng painterResource để lấy tài nguyên hình ảnh:

```
Image(painter = painterResource(id = R.drawable.my_image),
contentDescription = null)
```

Quản lý tài nguyên đúng cách giúp ứng dụng Android trở nên dễ bảo trì, có khả năng tùy chỉnh cao, và hỗ trợ nhiều nền tảng (ngôn ngữ, kích thước màn hình).

3.3. Xử lý sự kiện người dùng

Phần lớn các đối tượng đồ họa chúng ta đã học từ phần trước đều có khả năng đón nhận các tương tác từ phía người dùng, ví dụ như việc nhập thông tin vào TextField hay việc chọn Item từ một ChoiceGroup hay một List... Mỗi khi một tương tác xảy ra trên một thành phần giao diện, chương trình phải có những hành động trả lời tương ứng với những tương tác đó, ví dụ khi nhấn nút đăng nhập hay đăng xuất trên một website, hệ thống sẽ phải có những hành động để cho phép tham gia hay thoát khỏi hệ thống. Những tương tác xảy ra trên các thành phần giao diện như vậy chúng ta sẽ gọi là sự kiện và việc tạo ra những hành động trả lời cho các tương tác đó chúng ta sẽ gọi là xử lý sự kiện.

Khi đã thêm một số Views/Widget đến giao diện. Để được thông báo về UI events người dùng cần phải làm một trong hai điều: xác định một sự kiện lắng nghe và đăng ký nó với các View. Đây là cách lắng nghe sự kiện. Các class View có một phương pháp gọi là On<something>().

Ví dụ: View.OnClickListener (để xử lý nhấp chuột trên một View), View.OnTouchListener (để xử lý các sự kiện màn hình cảm ứng trong một View), và View.OnKeyListener(để xử lý thiết bị ép quan trọng trong một View). Vì vậy nếu muốn View được thông báo khi nó là clicked thực hiện và OnClickListener của nó gọi method onClick()(nơi thực hiện hành động sau khi nhấp chuột), và đăng ký nó vào xem với setOnClickListener().

Ghi đè một Callback method hiện cho View. Đây là những gì nên làm gì khi đã thực hiện lớp view của riêng bạn và muốn lắng nghe các sự kiện cụ thể xảy ra trong nó.

3.3.1. Đối tượng Button

Các Button có thể nhận biết được sự kiện click chuột trên giao diện từ người dùng thông qua một bộ lắng nghe sự kiện OnClickListener được gắn với chúng. Cách sử dụng đối tượng Button được minh hoạ qua đoạn code sau:

```
class Main : Activity(), View.OnClickListener {
    private lateinit var startButton: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        startButton = findViewById(R.id.start_button)
        startButton.setOnClickListener(this)
    }

    override fun onClick(v: View) {
        when (v.id) {
            R.id.start_button -> {
                // thực hiện công việc 1
            }
            R.id.some_other_button -> {
```

```
                // thực hiện công việc n
            }
        }
    }
}
```

Trong thực tế, khi viết phần mềm hiếm khi chỉ sử dụng một cách bắt sự kiện của của đối tượng Button mà có thể kết hợp sử dụng một trong năm cách dưới đây:

Cách 1. Sử dụng lớp thành viên: Cách này rất hữu ít khi chương trình có nhiều đối tượng Button nhưng mỗi đối tượng phải lắng nghe các sự kiện từ các bộ lắng nghe khác nhau. Trong đoạn chương trình mẫu dưới đây, lớp HandleClick là lớp thành viên của lớp Main.

```
class Main : Activity(), View.OnClickListener {
         lateinit var startButton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)
        startButton = findViewById(R.id.start_button)
        startButton.setOnClickListener(this)
    }

    override fun onClick(v: View) {
        when (v.id) {
            R.id.start_button -> {
                // thực hiện công việc 1
            }
            R.id.some_other_button -> {
                // thực hiện công việc n
            }
        }
    }
}
```

Cách 2. Sử dụng giao diện (Interface): khai báo một bộ lắng nghe OnClickListener và khởi tạo bằng cách sử dụng cú pháp new OnClickListener()....

Sử dụng giao điện là một bộ lắng nghe

```
class Main : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)

findViewById<Button>(R.id.button1).setOnClickListener(HandleClick())
    }

    private inner class HandleClick : View.OnClickListener {
        override fun onClick(arg0: View) {
            val btn = arg0 as Button
            val tv = findViewById<TextView>(R.id.textview1)
            tv.text = "Bạn vừa nhấn Button ${btn.text}"
        }
    }
}
```

Cách 3. Sử dụng lớp trong nặc danh (giấu tên): khai báo một giao diện OnClickListener bên trong lời gọi phương thức setOnClickListener(). Đây là cách được

sử dụng phổ biến và rất hữu ích khi mỗi bộ lắng nghe có những chức năng riêng mà không thể chia sẻ với các bộ lắng nghe khác. Tuy nhiên, cách này làm cho những người mới bắt đầu lập trình thấy khó hiểu khi đọc mã chương trình.

```
class Main : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)
        findViewById<Button>(R.id.button1).setOnClickListener {
            val btn = it as Button
            val tv = findViewById<TextView>(R.id.textview1)
            tv.text = "Bạn vừa nhấn Button ${btn.text}"
        }
    }
}
```

Cách 4. Hiện thực hoá giao diện OnClickListener từ lớp Activity:

với cách này, chính lớp Activity sẽ trở thành một bộ lắng nghe. Khi đối tượng Activity được thực thi, phương pháp này sẽ tiết kiệm được một không gian bộ nhớ bởi không đòi hỏi mỗi đối tượng con của lớp phải lưu trữ một phương thức onClick. Lớp Activity có thể hiện thực hoá nhiều giao diện bằng cách sử dụng dấu phẩy (,) phân cách các giao diện.

Hiện thực hoá trong lớp Activity

```
class Main : Activity(), View.OnClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)
        findViewById<Button>(R.id.button1).setOnClickListener(this)
    }

    override fun onClick(arg0: View) {
        val btn = arg0 as Button
        val tv = findViewById<TextView>(R.id.textview1)
        tv.text = "You pressed ${btn.text}"
    }
}
```

Cách 5: Sử dụng thuộc tính android:onClick trong file layout.

Sử dụng thuộc tính textitandroid:onClick

```
class Main : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)
    }

    fun handleClick(arg0: View) {
        val btn = arg0 as Button
        val tv = findViewById<TextView>(R.id.textview1)
        tv.text = "You pressed ${btn.text}"
    }
}
```

Trong file xml đối tượng Button khai báo thuộc tính an- droid:onClick là tên phương thức được thực thi khi click chuột lên Button.

```
<Button
```

```
    android:id="@+id/button1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button 1"
    android:onClick="HandleClick" />
```

Cách 6. Sử dụng hàm lambda

Lambda là một tính năng giúp viết code ngắn gọn và dễ đọc hơn. Khi xử lý sự kiện người dùng (ví dụ: nhấn nút, nhập liệu), thay vì sử dụng các lớp hoặc interface dài dòng, ta có thể sử dụng lambda để đơn giản hóa code.

Ví dụ khi xử lý sự kiện nhấn nút, cách viết truyền thống là:

```
button.setOnClickListener(object : View.OnClickListener {
    override fun onClick(v: View?) {
        Toast.makeText(this@MainActivity, "Button Clicked!",
        Toast.LENGTH_SHORT).show()
    }
})
```

Sử dụng lambda, có thể rút gọn thành:

```
button.setOnClickListener {
    Toast.makeText(this, "Button Clicked!", Toast.LENGTH_SHORT).show()
}
```

Kotlin tự động hiểu rằng phương thức onClick(View v) của interface View.OnClickListener chỉ có một tham số View, nên có thể bỏ qua khai báo View.

Đối với sự kiện setOnLongClickListener, cách viết truyền thống là:

```
button.setOnLongClickListener(object : View.OnLongClickListener {
    override fun onLongClick(v: View?): Boolean {
        Toast.makeText(this@MainActivity, "Long Click Detected!",
        Toast.LENGTH_SHORT).show()
        return true
    }
})
```

Sử dụng lambda:

```
button.setOnLongClickListener {
    Toast.makeText(this, "Long Click Detected!",
    Toast.LENGTH_SHORT).show()
    true // Trả về true để xác nhận sự kiện đã xử lý
}
```

Lambda giúp giảm bớt code boilerplate, dễ đọc và dễ hiểu hơn, đồng thời tối ưu hiệu suất vì không cần tạo class ẩn danh không cần thiết. Khi lập trình ứng dụng Android bằng Kotlin, nên ưu tiên sử dụng lambda để tối ưu code và cải thiện trải nghiệm lập trình.

Bốn cách sử dụng đầu tiên có thể sử dụng cho các sự kiện khác nhau như onLong Click, onKey, onTouch, onCreateContextMenu, onFocusChange, nhưng cách thứ năm

chỉ áp dụng cho sự kiện onClick. Với cách thứ năm thì trong phần mã chương trình java không cần sử dụng lệnh findViewById và setOnClickListener cho từng nút nhấn.

Thiết kế giao diện gồm một TextView và ba Button như hình 0 – 17, khi nhấn vào mỗi Button thì hiển thị nhãn của Button vừa chọn như dòng "Bạn vừa chọn 'nhãn Button được chọn":

File activity_main.xml có nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textview1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Click a button."
        android:textSize="20sp" />

    <LinearLayout
        android:orientation="horizontal"
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <Button
            android:id="@+id/button1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Button 1"
            android:onClick="HandleClick" />

        <Button
            android:id="@+id/button2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Button 2"
            android:onClick="HandleClick" />

        <Button
            android:id="@+id/button3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Button 3"
            android:onClick="HandleClick" />
    </LinearLayout>
</LinearLayout>
```

3.3.2. Đối tượng EditText

Để lắng nghe các sự kiện nhập liệu trên đối tượng EditText người lập trình phải hiện thực hoá giao diện lắng nghe TextWatcher đồng thời viết đề một trong các sự kiện before TextChanged(), onTextChanged() và afterTextChanged().

Một bộ lắng nghe sự kiện được gắn liền với một EditText qua phương thức addTextChangedListener().

Viết chương trình kiểm tra dữ liệu nhập vào đối tượng EditText có phải là số lớn hơn 100 hay không, nếu số nhập vào lớn hơn 100 thì thay thế số đó là 100. Ngoại lệ sẽ sinh ra nếu dữ liệu nhập vào không phải là số.

File main.xml chứa giao diện như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="Hãy nhập một số (0-100):" />

    <EditText
        android:id="@+id/percent"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="0"
        android:maxLength="3"
        android:inputType="number" />
</LinearLayout>
```

File MainActivity.java chứa nội dung như sau:

```
class MainActivity : Activity() {
    private lateinit var edit: EditText
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)
        edit = findViewById(R.id.percent)
        edit.addTextChangedListener(percentWatcher)
    }

    private val percentWatcher = object : TextWatcher {
        override fun afterTextChanged(s: Editable) {
            try {
                Log.d("Percentage", "input: $s")
                if (s.toString().toInt() > 100) {
                    s.replace(0, s.length, "100")
                }
            } catch (nfe: NumberFormatException) {
                Log.d("Lỗi:", "Dữ liệu nhập không phải số")
            }
        }

        override fun beforeTextChanged(s: CharSequence, start: Int,
        count: Int, after: Int) {}

        override fun onTextChanged(s: CharSequence, start: Int,
        before: Int, count: Int) {}
    }
}
```

3.3.3. Đối tượng CheckBox

Để lắng nghe sự kiện chọn CheckBox thì người lập trình phải gắn bộ lắng nghe sự kiện CheckBox.OnClickListener lên CheckBox và viết đè phương thức onClick().

Ngoài ra, CheckBox có thể được gắn bộ lắng nghe CompoundButton. OnCheckedChangeListener và viết đè phương thức onCheckedChanged().

Ví dụ 2-9. Đối tượng CheckBox

```
 class CheckBoxDemo : Activity(),
CompoundButton.OnCheckedChangeListener {
    private lateinit var cb: CheckBox

    override fun onCreate(icicle: Bundle?) {
        super.onCreate(icicle)
        setContentView(R.layout.main)
        cb = findViewById(R.id.check)
        cb.setOnCheckedChangeListener(this)
    }

    override fun onCheckedChanged(buttonView: CompoundButton,
      isChecked: Boolean) {
        cb.text = if (isChecked) {
            "This checkbox is: checked"
        } else {
            "This checkbox is: unchecked"
        }
    }
}
```

Sự kiện onClick() cho các CheckBox sẽ được trình bày rõ hơn trong ví dụ về các đối tượng lựa chọn.

3.3.4. Đối tượng Radio Button

Để lắng nghe sự kiện chọn trên các RadioButton thì chúng phải được gắn các bộ lắng nghe sự kiện. Mỗi RadioButton có thể ngắn bộ lắng nghe RadioGroup.OnClickListener và viết đè phương thức onClick(). Ngoài ra, người lập trình có thể xử lý sự kiện lựa chọn trên các đối tượng RadioButton thông đối tượng cha của nó là RadioGroup. Trong trường hợp này các RadioGroup phải được khai báo định danh trong file XML.

Các phương thức thường dùng của đối tượng RadioGroup trong chương trình java bao gồm:

- .check(): thiết lập trạng thái checked cho Radio Button thông qua id (ví dụ: group.check(R.id.radio1))
- .clearCheck(): hủy chọn tất cả các Radio trong nhóm
- .getCheckedRadioButtonId(): lấy về id của Radio Button đang được chọn (nếu không có Radio nào được chọn thì phương thức trả về giá trị -1).

Tạo giao diện gồm một TextView và một RadioGroup chứa ba Radio Button RadioGroup được gắn bộ lắng nghe sự kiện OnCheckedChangeListener để mỗi khi

người dùng chọn một Radio Button trên giao diện thì hiển thị nhãn tương ứng trên đối tượng TextView.

```
class RadioDemo : Activity(), RadioGroup.OnCheckedChangeListener {
    private lateinit var rg: RadioGroup
    private lateinit var tv: TextView

    override fun onCreate(c: Bundle?) {
        super.onCreate(c)
        setContentView(R.layout.activity_radio_demo)
        rg = findViewById(R.id.group)
        rg.check(R.id.radio2)
        rg.setOnCheckedChangeListener(this)
        tv = findViewById(R.id.tv)
    }

    override fun onCheckedChanged(group: RadioGroup, checkedId: Int) {
        // Implement your logic here
    }
}
```

Viết đè sự kiện bắt buộc onCheckedChanged, tham số id chứa định danh của Radio Button được chọn và được so sánh lần lượt với các id của từng Radio Button đ ểxác định đối tượng nào được chọn

```
override fun onCheckedChanged(rg: RadioGroup, id: Int) {
    when (id) {
        R.id.radio1 -> tv.text = "Bạn vừa chọn HĐH Android"
        R.id.radio2 -> tv.text = "Bạn vừa chọn HĐH iOS"
        R.id.radio3 -> tv.text = "Bạn vừa chọn HĐH WindowsPhone"
    }
}
```

3.3.5. Đối tượng Spinner

spin.setAdapter (aa); thiết lập Adapter cho Spinner Để lắng nghe sự kiện lựa chọn của người dùng thì mỗi Spinner phải được gắn một bộ lắng nghe sự kiện Adapter-View.OnItemSelectedListener và viết đè một trong các phương thức bắt buộc là onItemSelected() hoặc onNothingSelected(). Phương thức onItemSelected() được thực thi khi người dùng lựa chọn một phần tử (gọi là item) từ danh sách của Spinner. Phương thức on- NothingSelected() được thực thi khi người dùng không chọn item nào từ danh sách.

Tạo giao diện gồm hai đối tượng TextView và một đối tượng Spinner như hình 0 – 21. Đối tượng Spinner chứa danh sách các hằng số lựa chọn về tháng trong năm. Chương trình java thực hiện xử lý khi có một item trong Spinner được chọn thì hiển thị item đó lên TextView, nếu không có Item nào được chọn thì giá trị hiển thị trên TextView là trống.

```
File SpinnerDemo.java chứa nội dung như sau:
 class SpinnerDemo : Activity(), AdapterView.OnItemSelectedListener {
    private lateinit var selection: TextView
    private lateinit var spin: Spinner
    private val items = arrayOf(
        "January", "February", "March", "April",
```

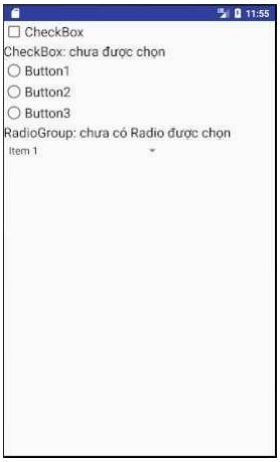
```
        "May", "June", "July", "August",
        "September", "October", "November", "December"
    )

    override fun onCreate(icicle: Bundle?) {
        super.onCreate(icicle)
        setContentView(R.layout.main)

        selection = findViewById(R.id.selection)
        spin = findViewById(R.id.spinner)
        spin.onItemSelectedListener = this

        val adt = ArrayAdapter(this,
android.R.layout.simple_spinner_item, items)

adt.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_i
tem)
        spin.adapter = adt
    }

    override fun onItemSelected(parent: AdapterView<*>, v: View,
position: Int, id: Long) {
        selection.text = "Bạn vừa chọn tháng: ${items[position]}"
    }

    override fun onNothingSelected(parent: AdapterView<*>) {
        selection.text = ""
    }
}
```

File main.xml chứa nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="Hãy chọn một Item:" />

    <Spinner
        android:id="@+id/spinner"
        android:layout_width="match_parent"
        android:layout_height="80dp" />

    <TextView
        android:id="@+id/tv"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="" />
</LinearLayout>
```

Chương trình java xử lý sự kiện: khi người dùng chọn CheckBox thì hiển thị trạng thái của CheckBox lên TextView thứ nhất, khi người dùng chọn một trong các Radio Button thì hiển thị nhãn của Radio được chọn lên TextView thứ 2. CheckBox và các

Radio được gắn bộ lắng nghe sự kiện OnClickListener, Spinner được gắn bộ lắng nghe sự kiện OnItemSelectedListener.



Hình 3.7: Ví dụ tổng hợp về đối tượng Checkbox, Radio Button và Spinner

File main.xml chứa nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <CheckBox
        android:id="@+id/cbxBox1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:text="CheckBox"
        android:checked="false" />

    <TextView
        android:id="@+id/txtCheckBox"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:text="CheckBox: chưa được chọn" />

    <RadioGroup
        android:id="@+id/rgGroup1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <RadioButton
            android:id="@+id/RB1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="20sp"
            android:text="Button1" />

        <RadioButton
            android:id="@+id/RB2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
            android:textSize="20sp"
            android:text="Button2" />

        <RadioButton
            android:id="@+id/RB3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:textSize="20sp"
            android:text="Button3" />
    </RadioGroup>

    <TextView
        android:id="@+id/txtRadio"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:text="RadioGroup: chưa có Radio được chọn" />

    <Spinner
        android:id="@+id/spin"
        android:layout_width="250dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="2dp"
        android:textSize="30sp" />
</LinearLayout>
```

File SelectExample.java chứa nội dung như sau:

```
class SelectExample : Activity() {
    private lateinit var checkBox: CheckBox
    private lateinit var txtCheckBox: TextView
    private lateinit var txtRadio: TextView
    private lateinit var rb1: RadioButton
    private lateinit var rb2: RadioButton
    private lateinit var rb3: RadioButton
    private lateinit var spin: Spinner

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)

        checkBox = findViewById(R.id.cbxBox1)
        txtCheckBox = findViewById(R.id.txtCheckBox)
        txtRadio = findViewById(R.id.txtRadio)
        rb1 = findViewById(R.id.RB1)
        rb2 = findViewById(R.id.RB2)
        rb3 = findViewById(R.id.RB3)
        spin = findViewById(R.id.spin)

        checkBox.setOnClickListener {
            txtCheckBox.text = if (checkBox.isChecked) {
                "CheckBox: được chọn"
            } else {
                "CheckBox: chưa được chọn"
            }
        }

        rb1.setOnClickListener {
            txtRadio.text = "Radio: Button 1 được chọn"
        }

        rb2.setOnClickListener {
```

```
            txtRadio.text = "Radio: Button 2 được chọn"
        }

        rb3.setOnClickListener {
            txtRadio.text = "Radio: Button 3 được chọn"
        }

        val lsMusketeers = arrayListOf("Item 1", "Item 2", "Item 3")
        val aspnMusketeers = ArrayAdapter(this,
        android.R.layout.simple_spinner_item, lsMusketeers)
        aspnMusketeers.
      setDropDownViewResource(android.R.layout.simple_spinner_dropdown
_item
      )
        spin.adapter = aspnMusketeers

        spin.onItemSelectedListener = object
             : AdapterView.OnItemSelectedListener {
            override fun onNothingSelected(parent: AdapterView<*>) {}

            override fun onItemSelected(parent: AdapterView<*>,
            v: View, position: Int, id: Long) {
                // Implement any logic here for item selection
            }
        }
    }
}
```

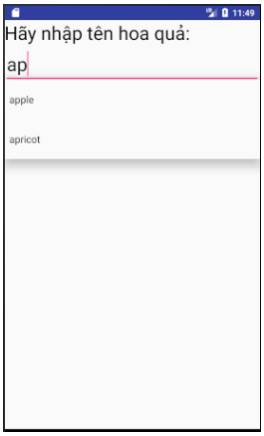
3.3.6. Đối tượng AutoCompleteTextView

Để lắng nghe sự kiện nhập liệu trên đối tượng AutoCompleteTextView, người lập trình phải hiện thực hoá giao diện TextWatcher và viết đè một trong các phương thức onTextChanged(), beforeTextChanged(), af- terTextChanged() tương tự như đối tượng EditText.

Phương thức addTextChangedListener() cho phép gắn bộ lắng nghe sự kiện vào đối tượng AutoCompleteTextView.

autocomplete = (AutoCompleteTextView)findViewById(R.id.autocomp

autocomplete.addTextChangedListener(this);



Hình 3.8: Minh hoạ đối tượng AutoCompleteTextView.

Danh sách các từ gợi ý khi nhập dữ liệu cho đối AutoCompleteTextView được lưu trong một đối tượng Adapter, sau đó được gắn vào đối tượng này sử dụng phương thức setAdapter(), cụ thể như sau:

```
val autocompleteItems = arrayOf("apple", "banana", "mango",
"pineapple", "apricot", "orange", "pear", "grapes")
autocomplete.setAdapter(ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, autocompleteItems))
```

Minh hoạ về hoạt động của đối tượng AutoCompleteTextView như hình trên. Đối tương AutoCompleteTextView cho phép hiện các từ gợi ý khi người dùng nhập tên loại hoa quả, sau khi kết thúc nhập một TextView sẽ hiển thị giá trị vừa chọn trên AutoCompleteTextView.

File main.xml chứa nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="Hãy nhập tên hoa quả:" />

    <AutoCompleteTextView
        android:id="@+id/autocomplete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:completionThreshold="2" />

    <TextView
        android:id="@+id/field"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="30sp"
        android:text="" />
</LinearLayout>
```

File AutoComplete.java chứa nội dung như sau:

```
class AutoComplete : Activity(), TextWatcher {
    private lateinit var field: TextView
    private lateinit var autocomplete: AutoCompleteTextView
    private val autocompleteItems = arrayOf("apple", "banana", "mango",
    "pineapple", "apricot", "orange", "pear", "grapes")

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.main)

        field = findViewById(R.id.field)
        autocomplete = findViewById(R.id.autocomplete)
        autocomplete.addTextChangedListener(this)
        autocomplete.setAdapter(ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, autocompleteItems))
```

```
        }

        override fun onTextChanged(arg0: CharSequence, arg1: Int, arg2: Int,
        arg3: Int) {
            field.text = autocomplete.text
        }

        override fun beforeTextChanged(s: CharSequence, start: Int, count:
    Int,
        after: Int) {}

        override fun afterTextChanged(s: Editable) {}
    }
```

Bài tập bổ trợ

Bài tập thực hành cơ bản

1. Thiết kế giao diện với EditText, Button và TextView. Khi nhấn nút, hiển thị nội dung nhập từ EditText lên TextView.
2. Viết ứng dụng sử dụng CheckBox để chọn sở thích và hiển thị lựa chọn khi nhấn Button.
3. Tạo giao diện với RadioGroup để chọn giới tính và hiển thị lựa chọn khi nhấn Button.
4. Sử dụng Spinner để chọn quốc gia và hiển thị quốc gia được chọn.
5. Tạo ứng dụng cho phép người dùng nhập số trong EditText và tính tổng khi nhấn nút.

Bài giải mẫu

Bài 1: Xử lý sự kiện `Button`

```
button.setOnClickListener {
    val name = editText.text.toString()
    textView.text = "Xin chào, $name!"
}
```

Bài 2: Hiển thị nội dung `EditText`

```
button.setOnClickListener {
    val input = editText.text.toString()
    textView.text = "Bạn đã nhập: $input"
}
```